🐾 Pet Care System

A Complete Guide for Students

Understanding a Full-Stack Web Application

📋 Table of Contents

1. System Overview
2. Architecture
3. User Roles
4. Database Structure
5. Key Features
6. How The System Works
7. Frontend Architecture
8. Backend Architecture
9. API Endpoints
10. Security & Authentication

1. System Overview

What is Pet Care?

Pet Care is a comprehensive **web application** designed to help pet owners manage their pets' health, book veterinary appointments, access educational content, and connect with veterinarians.

> **Key Features:**
>
> - 🐕 Manage multiple pets and their health records
> - 📅 Book appointments with veterinarians
> - 💬 AI-powered health chatbot for quick advice
> - 🏥 Request professional consultations with image uploads
> - 📚 Access educational pet care content
> - 🐾 Browse pets available for adoption
> - 📍 Find nearby veterinarians using maps
> - 🔐 Secure user authentication and role-based access

Technology Stack

Frontend

- React 19
- TypeScript
- React Router v7
- Axios
- Leaflet Maps
- Recharts

Backend

- Java 17
- Spring Boot 3.4.3
- Spring Security
- JPA/Hibernate
- MySQL Database
- JWT Authentication

Integrations

- Groq API (Chatbot)
- Gemini API (Vision)
- JWT (Auth)
- BCrypt (Encryption)

Deployment

- Docker
- Docker Compose
- localhost:3000 (Frontend)
- localhost:8080 (Backend)

---

2. System Architecture

Overall System Design

```
┌──────────────────────────────────────────────────────────────────┐
│                      CLIENT (BROWSER)                            │
│  React Frontend Application (Port 3000)                          │
│  - Pages, Components, State Management                           │
└──────────────────────────────┬───────────────────────────────────┘
                               │
                               │
                      HTTP REST API
                               │
┌──────────────────────────────▼───────────────────────────────────┐
│                BACKEND SERVER (PORT 8080)                        │
│            Spring Boot REST API                                  │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │ Controllers → Services → Repositories                    │   │
│   └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────┬───────────────────────────────────┘
                               │
                               │
                            JDBC
                               │
┌──────────────────────────────▼───────────────────────────────────┐
│                 DATABASE (MySQL)                                 │
│  Tables: Users, Pets, Appointments, Consultations, etc.          │
└──────────────────────────────────────────────────────────────────┘
```

Three-Tier Architecture Layers

1. **Presentation Layer (Frontend)** - React components and UI
2. **Business Logic Layer (Backend Services)** - Spring Boot services
3. **Data Access Layer (Database)** - MySQL with JPA/Hibernate

Each layer communicates with the layer below it, creating separation of concerns and making the code maintainable.

3. User Roles & Access Control

Three Types of Users

👤 USER (Pet Owner)

- Register and manage account
- Add and manage multiple pets
- Book appointments with veterinarians
- Request professional consultations
- Chat with AI health assistant
- View educational content
- Browse adoption listings
- Find nearby vets

⚕️ VET (Veterinarian)

- Register as veterinarian
- Accept and manage appointments
- Review consultation requests
- Provide professional diagnoses
- Manage schedule
- View patient history

🛡️ ADMIN (Administrator)

- Manage all users
- Manage system content
- Monitor consultations
- View system statistics

- System configuration

Role-Based Access Control (RBAC)

```
Endpoint Security:
├─ /api/auth/** → PUBLIC (anyone)
├─ /dashboard → PROTECTED (all authenticated users)
├─ /consultations/{id}/review → VET ONLY
├─ /admin/** → ADMIN ONLY
└─ /pets → OWNER (own pets only)
```

---

4. Database Structure

Key Database Tables

Users Table

Column	Type	Description
id	INT (PK)	Auto-generated primary key
firstName	VARCHAR	User's first name
email	VARCHAR (UNIQUE)	Email address (login identifier)
password	VARCHAR	Encrypted password
role	ENUM	USER, VET, or ADMIN

Pets Table

Column	Type	Description
id	INT (PK)	Auto-generated ID
name	VARCHAR	Pet's name
species	VARCHAR	Dog, Cat, Bird, etc.
owner_id	INT (FK)	Links to Users table

Appointments Table

Column	Type	Description
id	INT (PK)	Auto-generated ID
pet_id	INT (FK)	Which pet
vet_id	INT (FK)	Assigned veterinarian
appointmentTime	TIMESTAMP	Date and time of appointment
status	ENUM	SCHEDULED, COMPLETED, CANCELLED

Consultations Table

Column	Type	Description
id	INT (PK)	Auto-generated ID
pet_id	INT (FK)	Pet requesting consultation
userDescription	TEXT	Symptoms/concerns from owner
imageUrl	VARCHAR	URL to uploaded image
aiPreliminaryDiagnosis	TEXT	AI analysis from Gemini API
status	ENUM	PENDING, REVIEWED, COMPLETED

> 💡 **Learning Point:** Tables are related through Foreign Keys (FK). For example, Pets.owner_id points to Users.id, creating a relationship where each pet belongs to exactly one user.

5. Key Features Explained

1️⃣ User Authentication & Security

Users create accounts with email and password. The system uses `JWT tokens` to keep users logged in securely. Passwords are encrypted using `BCrypt` so even administrators cannot see them.

2️⃣ Pet Management

Users can add multiple pets, track their age, weight, species, breed, and medical history. The system stores all this information in the database linked to the pet owner.

3️⃣ AI-Powered Chatbot

Uses the **Groq API** (powered by Llama-3.3-70b model) to provide quick health advice. Perfect for owners who want initial guidance before visiting a vet. Responses are kept short and friendly.

4️⃣ Professional Consultation System

A three-stage process:

1. **Step 1:** Owner describes symptoms and uploads a photo
2. **Step 2:** AI (Gemini API) analyzes image and provides preliminary diagnosis
3. **Step 3:** Licensed vet reviews and provides professional feedback

5️⃣ Appointment Booking

Users browse available vets, select a time slot, and book appointments. The system manages scheduling and prevents conflicts.

6️⃣ Educational Content

Articles and guides about pet care, health tips, breed-specific advice, and preventive care measures.

7️⃣ Adoption Listings

Browse available pets for adoption with detailed information about each pet.

8️⃣ Location Services

Uses **Leaflet Maps** to show nearby veterinarians and calculates distances.

9️⃣ Admin Dashboard

Administrators can view all users, appointments, consultations, and manage system content.

🔟 Notifications

Users receive notifications about appointment confirmations, consultation status updates, and other important events.

---

6. How The System Works - Step by Step

Complete User Journey: Registration → Pet Management → Booking Appointment

Step 1: User Registration

```
User Opens Website
    ↓
Clicks "Register"
    ↓
Fills Form:
  - First Name: "John"
  - Last Name: "Smith"
  - Email: "john@email.com"
  - Password: "SecurePass123"
    ↓
Frontend sends to Backend:
  POST /api/auth/register
  {
    "firstName": "John",
    "lastName": "Smith",
    "email": "john@email.com",
    "password": "SecurePass123",
    "role": "USER"
  }
    ↓
Backend (AuthService):
  - Validates input is not empty
  - Checks email not already registered
  - Hashes password using BCrypt
  - Creates User record in database
  - Generates 24-hour JWT token
    ↓
Backend Response:
  {
    "token": "eyJhbGciOiJIUzI1NiIs...",
    "user": {
```

```
      "id": 1,
      "firstName": "John",
      "email": "john@email.com",
      "role": "USER"
    }
  }
    ↓
Frontend receives response:
  - Saves token to localStorage
  - Saves user info to localStorage
  - Updates global AuthContext
  - Redirects to Dashboard
    ↓
✅ User is now logged in!
```

Step 2: Adding a Pet

```
User clicks "Add Pet" on Dashboard
    ↓
PetsPage component loads
    ↓
User fills form:
  - Name: "Buddy"
  - Species: "Dog"
  - Breed: "Golden Retriever"
  - Age: 3
  - Weight: 30 kg
  - Medical History: "Vaccinated, Allergic to chicken"
    ↓
User clicks "Save Pet"
    ↓
Frontend sends:
  POST /api/pets
  Headers: {
    "Authorization": "Bearer eyJhbGciOiJIUzI1NiIs..."
  }
  Body: {
    "name": "Buddy",
    "species": "Dog",
    "breed": "Golden Retriever",
    "age": 3,
```

```
    "weight": 30,
    "medicalHistory": "Vaccinated, Allergic to chicken"
  }
    ↓
Backend JwtAuthFilter:
  - Extracts token from header
  - Validates token signature
  - Checks token not expired
  - Extracts user email from token
    ↓
Backend PetController:
  - Receives PetRequest
  - Calls PetService.createPet()
    ↓
Backend PetService:
  - Finds User by email
  - Creates Pet object
  - Sets owner = user
  - Calls petRepository.save()
    ↓
Database:
  INSERT INTO pets
  (name, species, breed, age, weight, medicalHistory, owner_id)
  VALUES ('Buddy', 'Dog', 'Golden Retriever', 3, 30, '...', 1)
    ↓
Database returns: Pet with id=5
    ↓
Backend returns:
  {
    "id": 5,
    "name": "Buddy",
    "species": "Dog",
    "breed": "Golden Retriever",
    "age": 3,
    "weight": 30,
    "medicalHistory": "Vaccinated, Allergic to chicken",
    "owner": { "id": 1 }
  }
    ↓
Frontend receives and updates list
    ↓
```

```
User sees: "Pet added successfully!"
✅ Pet is now in system!
```

Step 3: Booking an Appointment

```
User clicks "Book Appointment"
    ↓
AppointmentsPage loads
    ↓
User selects:
  - Pet: "Buddy" (id: 5)
  - Vet: "Dr. Sarah Patel" (id: 3)
  - Date/Time: "2025-03-10 2:00 PM"
  - Reason: "Annual checkup"
    ↓
Frontend sends:
  POST /api/appointments
  {
    "petId": 5,
    "vetId": 3,
    "appointmentTime": "2025-03-10T14:00:00",
    "reason": "Annual checkup"
  }
    ↓
Backend AppointmentService:
  - Validates pet exists and belongs to user
  - Validates vet exists and has role VET
  - Checks for scheduling conflicts
  - Creates Appointment record:
    appointment.setPet(pet)
    appointment.setOwner(user)
    appointment.setVet(vet)
    appointment.setAppointmentTime(...)
    appointment.setReason("Annual checkup")
    appointment.setStatus("SCHEDULED")
    ↓
Database:
  INSERT INTO appointments
  (pet_id, owner_id, vet_id, appointmentTime, reason, status)
  VALUES (5, 1, 3, '2025-03-10 14:00:00', 'Annual checkup', 'SCHEDULED')
    ↓
```

```
Backend returns appointment details
    ↓
Frontend displays:
  "Appointment booked!
   Buddy with Dr. Sarah Patel on Mar 10, 2025 at 2:00 PM"
✅ Appointment is confirmed!
```

Step 4: Using AI Chatbot for Health Advice

```
User goes to ChatPage
    ↓
Types: "My dog has been scratching a lot"
    ↓
Frontend sends:
  POST /api/chatbot/chat
  {
    "message": "My dog has been scratching a lot"
  }
    ↓
Backend ChatAssistantService:
  - Takes user message
  - Adds system prompt (act as friendly pet health chatbot)
  - Calls Groq API at https://api.groq.com/openai/v1/chat/completions
  - Uses model: llama-3.3-70b-versatile
  - Request includes API key for authentication
    ↓
Groq API (Cloud):
  - Processes prompt with Llama model
  - Generates response
  - Returns: "Scratching in dogs can indicate allergies, fleas, or dry skin
             I'd recommend checking for fleas and consulting your vet soon!
    ↓
Backend receives response
    ↓
Backend returns to Frontend
    ↓
Frontend displays chat message
    ↓
```

```
User reads: "Scratching in dogs can indicate..."
✅ User gets helpful advice instantly!
```

Step 5: Professional Consultation with Image Upload

```
User goes to Consultations page
    ↓
Clicks "Request Consultation"
    ↓
Form shows:
  - Select Pet (dropdown)
  - Describe symptoms
  - Upload image (optional)
    ↓
User fills:
  Pet: "Buddy"
  Description: "Red rash on paws, excessive licking"
  Uploads: photo_of_rash.jpg
    ↓
Frontend uploads to Backend:
  POST /api/consultations
  {
    "petId": 5,
    "userDescription": "Red rash on paws, excessive licking",
    "imageUrl": "uploads/photo_of_rash.jpg"
  }
    ↓
Backend ConsultationService:
  - Validates pet exists and belongs to user
  - Saves image file to server
  - Sends description + image to Gemini API
    ↓
Gemini API (Google Cloud):
  - Analyzes image (vision AI)
  - Sees inflammation on paws
  - Considers description
  - Returns: "Preliminary diagnosis: Possible allergic reaction
              or contact dermatitis. Recommend hypoallergenic
              products and observation for 7 days."
    ↓
```

```
Backend stores Consultation:
  id: 1, pet: Buddy, userDescription: "Red rash...",
  imageUrl: "...", status: "PENDING",
  aiPreliminaryDiagnosis: "Possible allergic..."
    ↓
Notification sent to all VETs
    ↓
Frontend shows: "Consultation submitted! A vet will review soon."
    ↓
User Dashboard shows: "Your consultation is pending review"
✅ Consultation in system!
```

Step 6: Vet Reviews Consultation

```
Vet logs in (role: VET)
    ↓
Goes to /consultations
    ↓
Sees list of PENDING consultations
    ↓
Clicks on "Red rash on paws" consultation
    ↓
Reviews:
  - Pet info: Buddy, Golden Retriever, 3 years old
  - Owner description: "Red rash on paws, excessive licking"
  - Photo of rash
  - AI preliminary diagnosis: "Possible allergic reaction..."
    ↓
Vet provides professional feedback:
  "The rash appears to be contact dermatitis from bathing products.
   I recommend using hypoallergenic shampoo and observing for
   7-10 days. If not improving, bring Buddy to clinic for testing."
    ↓
Vet submits:
  PUT /api/consultations/1/review
  {
    "vetFeedback": "The rash appears to be contact dermatitis...",
    "status": "REVIEWED"
  }
    ↓
Backend updates Consultation:
```

```
  consultation.vetFeedback = "The rash appears..."
  consultation.status = "REVIEWED"
  consultation.vet_id = 3 (vet's ID)
    ↓
Database updates consultation record
    ↓
Notification sent to pet owner
    ↓
Owner receives: "Dr. Sarah reviewed your consultation!"
    ↓
Owner can now view vet's professional feedback
✅ Consultation reviewed and completed!
```

7. Frontend Architecture

Technology Stack

- **React 19** - UI library
- **TypeScript** - Type safety
- **React Router DOM** - Page navigation
- **Axios** - HTTP requests
- **Leaflet** - Interactive maps
- **Recharts** - Data visualizations

Folder Structure

```
pet-care-frontend/src/
├── App.tsx                    # Main app with all routes
├── context/
│   └── AuthContext.tsx        # Global authentication state
├── components/
│   ├── AppLayout.tsx          # Sidebar + TopBar wrapper
│   ├── Navbar.tsx             # Top navigation
│   ├── ProtectedRoute.tsx     # Require login
│   └── ui/
│       ├── LoadingSpinner.tsx
│       └── Toast.tsx          # Notifications
├── pages/
│   ├── LandingPage.tsx        # Home page
│   ├── LoginPage.tsx          # Login form
│   ├── RegisterPage.tsx       # Sign up form
│   ├── DashboardPage.tsx      # Main dashboard
│   ├── PetsPage.tsx           # Manage pets
│   ├── AppointmentsPage.tsx   # Book appointments
│   ├── ConsultationPage.tsx   # Request consultation
│   ├── ChatbotPage.tsx        # AI chat
│   └── ... (other pages)
```

```
└── api/
    └── axiosConfig.ts        # HTTP client with auth
```

Key Concepts

🔐 AuthContext - Global Authentication State

A React Context that manages login state globally across the app. Any component can access:

- `user` - Current logged-in user info
- `token` - JWT token for API requests
- `login()` - Function to authenticate
- `logout()` - Function to sign out

🛡️ ProtectedRoute - Access Control

A wrapper component that checks if user is logged in. If not, redirects to login page.

📱 AppLayout - Page Structure

Wraps protected pages with sidebar navigation and top header bar. Structure:

```
┌──────────────────────────┐
│       Top Bar            │
├──────────┬───────────────┤
│          │               │
│ Sidebar  │ Page Content  │
│          │               │
│          │               │
└──────────┴───────────────┘
```

📡 Axios - HTTP Client

Automatically adds JWT token to every API request header:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Data Flow Example

```
User visits /pets page
    ↓
React Router renders PetsPage component
    ↓
PetsPage useEffect hook runs:
  - Gets token from AuthContext
  - Calls axios.get('/api/pets')
    ↓
Axios adds Authorization header:
  Authorization: Bearer {token}
    ↓
Backend validates token and returns pet list
    ↓
PetsPage stores data in useState
    ↓
Component renders pets in table/list
    ↓
User clicks "Add Pet" → form opens
    ↓
User fills form and submits
    ↓
axios.post('/api/pets', petData)
    ↓
Backend creates pet and returns it
    ↓
Frontend receives response
    ↓
Frontend updates state
    ↓
Component re-renders with new pet in list
    ↓
User sees: "Pet added successfully!"
    ↓
New pet appears in list
```

8. Backend Architecture

Technology Stack

- **Java 17** - Programming language
- **Spring Boot 3.4.3** - Web framework
- **Spring Security** - Authentication
- **Spring Data JPA** - Database access
- **JWT** - Token-based auth
- **BCrypt** - Password encryption
- **MySQL** - Database

Folder Structure

```
src/main/java/com/petcare/backend/
├── BackendApplication.java         # Entry point
├── config/
│   ├── SecurityConfig.java        # JWT + Security setup
│   ├── GlobalExceptionHandler.java# Error handling
│   └── DataInitializer.java       # Test data
├── controller/                     # REST endpoints
│   ├── AuthController.java        # /api/auth/**
│   ├── PetController.java         # /api/pets/**
│   ├── AppointmentController.java
│   ├── ConsultationController.java
│   ├── ChatbotController.java
│   └── ... (other controllers)
├── service/                        # Business logic
│   ├── AuthService.java           # Register, login
│   ├── PetService.java            # CRUD operations
│   ├── ConsultationService.java   # Consultation logic
│   ├── ChatAssistantService.java  # Groq API calls
│   └── ... (other services)
├── dto/                            # Data Transfer Objects
```

```
│   ├── AuthResponse.java
│   ├── ConsultationDTO.java
│   └── ...
├── entity/                          # Database models
│   ├── User.java                   # @Entity table
│   ├── Pet.java
│   ├── Appointment.java
│   ├── Consultation.java
│   └── ...
├── repository/                      # Database access
│   ├── UserRepository.java         # extends JpaRepository
│   ├── PetRepository.java
│   ├── AppointmentRepository.java
│   └── ...
└── security/
    ├── JwtAuthFilter.java          # Validate tokens
    └── JwtTokenProvider.java       # Generate tokens
```